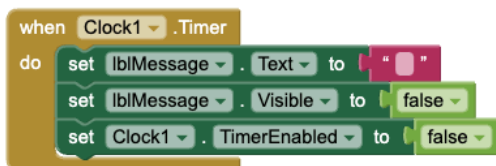
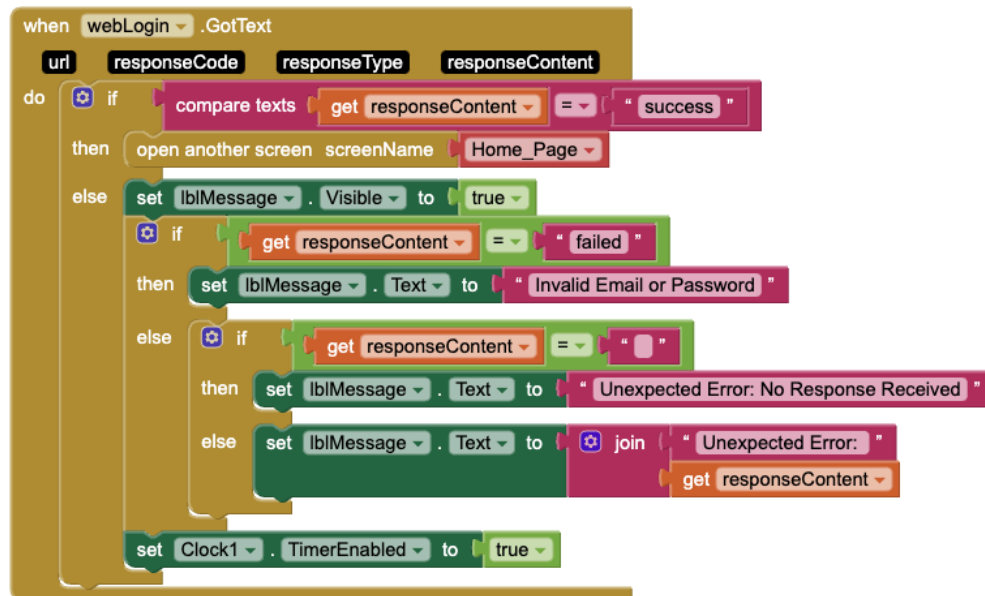
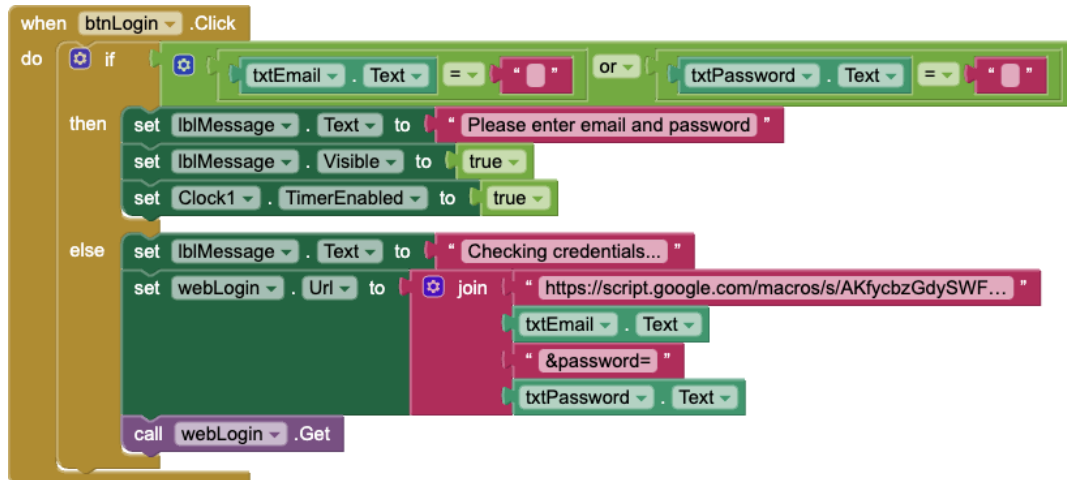


Appendix B:

Login



HOME

when Home_Img .Click

do open another screen screenName Home_Page

when IMG_NewStudent .Click

do open another screen screenName StudentRequests

when IMG_NewTutor .Click

do open another screen screenName TutorRequests

when BTN_NewStudent .Click

do open another screen screenName StudentRequests

when BTN_NewTuto .Click

do open another screen screenName TutorRequests

when Email_img .Click

do open another screen screenName SendBulkAutomatedEmails

when IMG_MatchedStudent .Click

do open another screen screenName MatchedStudentSection

when IMG_MatchedTutor .Click

do open another screen screenName MatchedTutorSection

when Email_Button .Click

do open another screen screenName SendBulkAutomatedEmails

when BTN_MatchedStudent .Click

do open another screen screenName MatchedStudentSection

when BTN_MatchedTutor .Click

do open another screen screenName MatchedTutorSection

New Students

```
initialize global fetchStudentList to create empty list

initialize global studentData to create empty list

initialize global displayList to create empty list

initialize global studentList to create empty list

initialize global selectedStudentID to 
initialize global selectedStudent to 
initialize global RowNumber to 1
initialize global rowIndex to 1

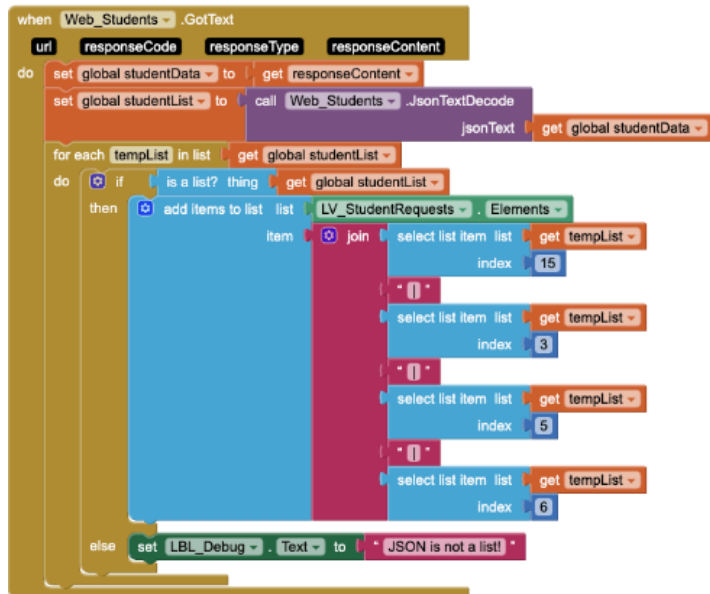
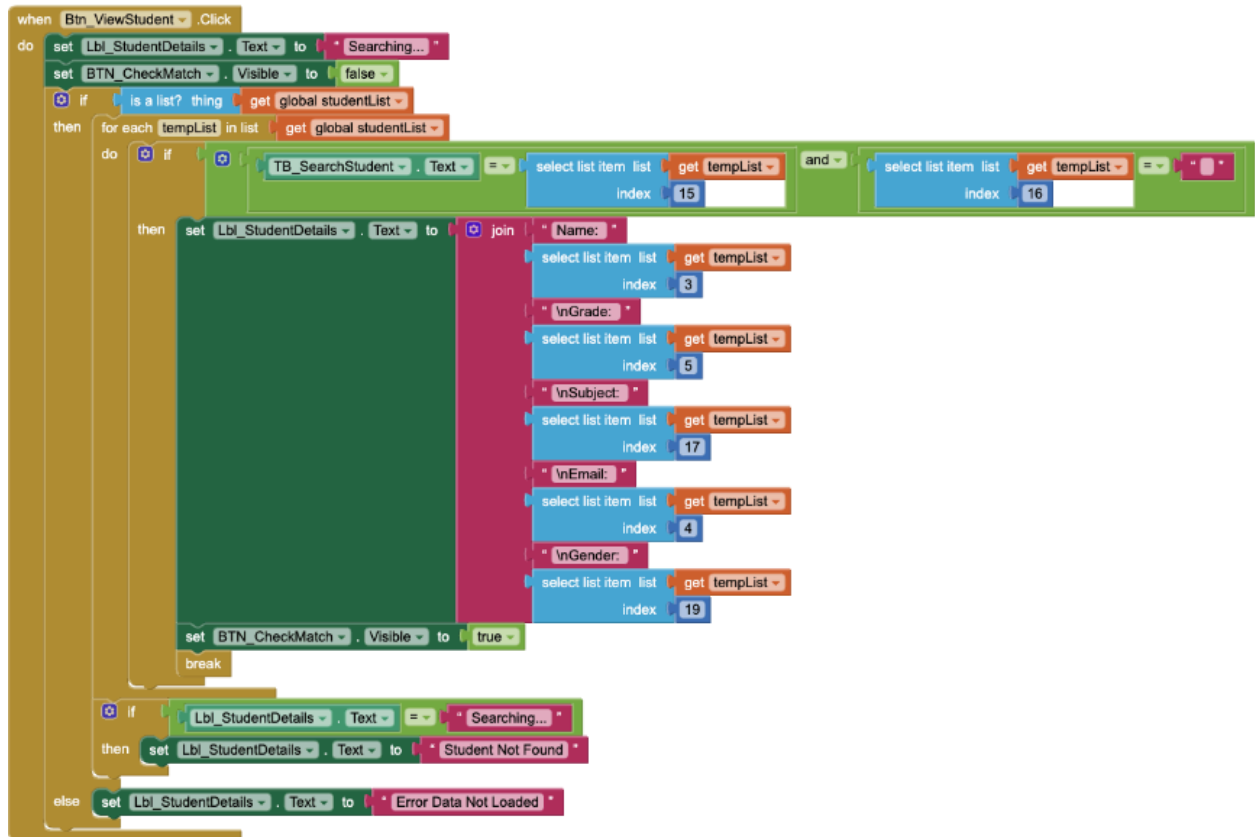
when Home_Img .Click
do open another screen screenName Home_Page

when BTN_CheckMatch .Click
do set global selectedStudentID to TB_SearchStudent . Text
   set global selectedStudent to Lbl_StudentDetails . Text
   open another screen with start value screenName MatchScreen
   startValue get global selectedStudentID

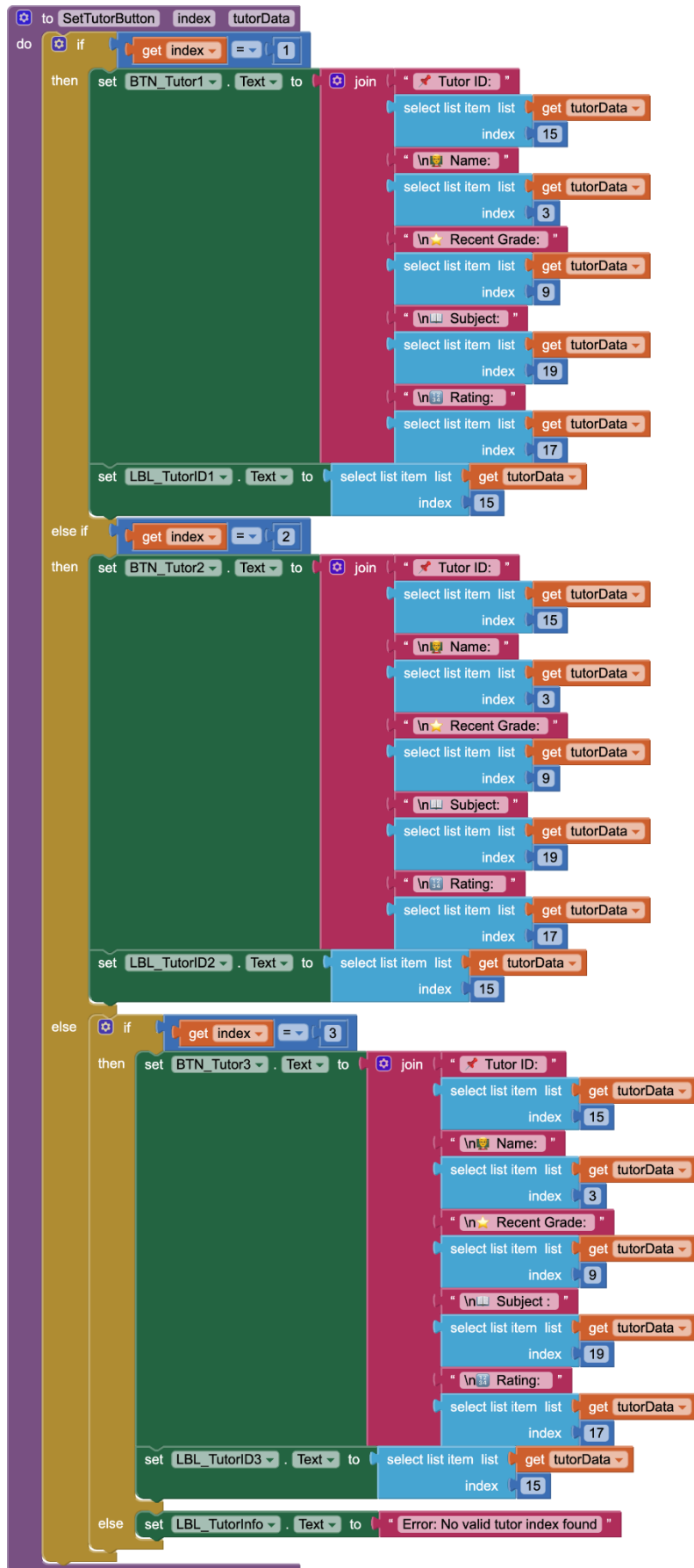
to FetchStudentData
do set Web_Students . Url to "https://script.google.com/macros/s/AKfycbw8hrL7a..."
   call Web_Students .Get

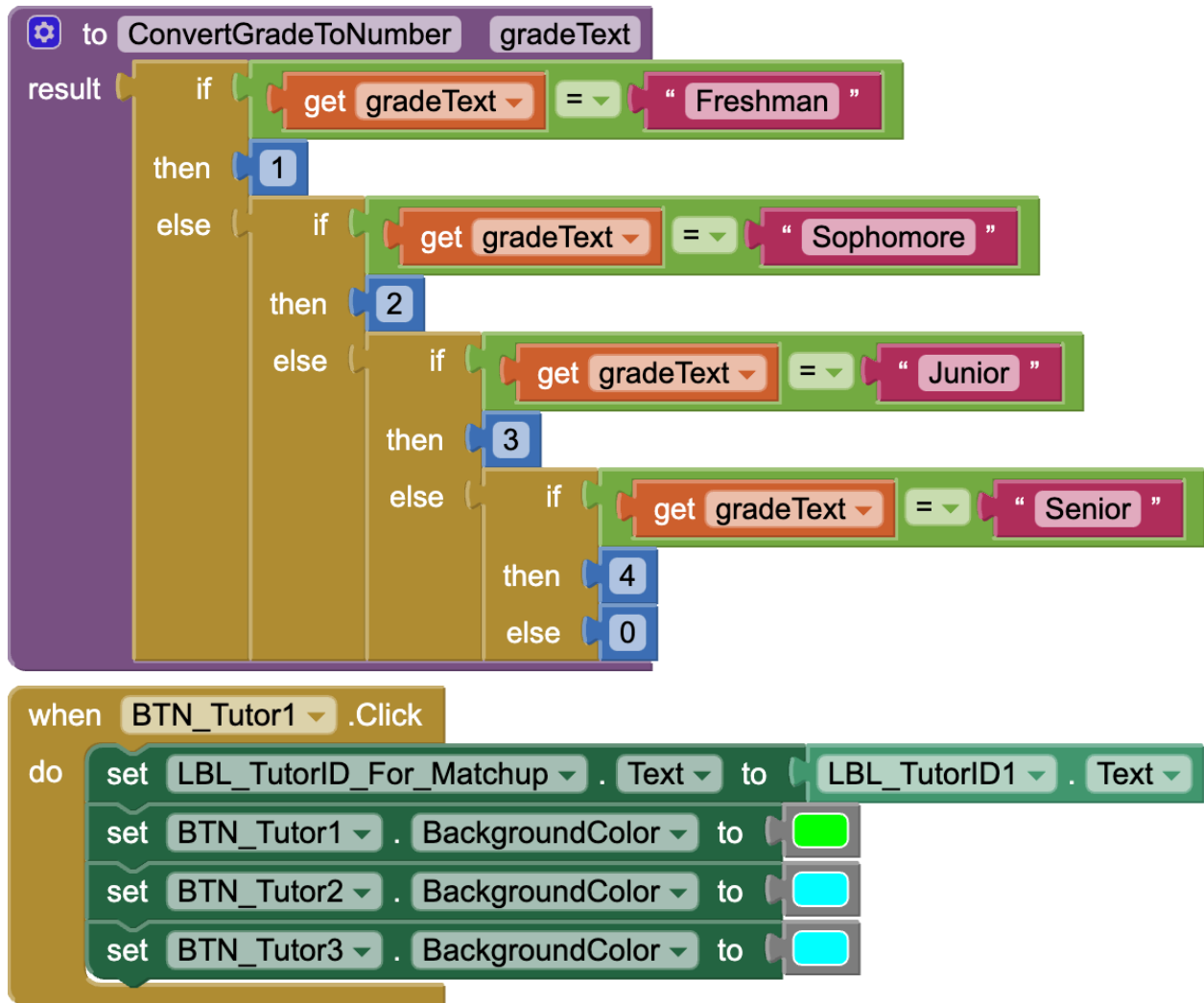
when StudentRequests .Initialize
do Web_FetchStudents . Url https://script.google.com/macros/s/AKfycbzMU300J...
   call Web_FetchStudents .Get
   call FetchStudentData

when Web_FetchStudents .GotText
url responseCode responseType responseContent
do set global fetchStudentList to list from csv table text get responseContent
   remove list item list get global fetchStudentList
   index 1
   initialize local dataToBeShown to create empty list
   in for each item in list get global fetchStudentList
   do if select list item list get item
        index 16
      then add items to list list get dataToBeShown
           item join select list item list get item
                    index 15
                    select list item list get item
                    index 3
                    select list item list get item
                    index 19
   set LV_StudentRequests . Elements to get dataToBeShown
```



Matchup Screen





```

when MatchScreen.Initialize
do
  set global selectedStudentID to get start value
  set global selectedStudentDetails to get start value
  set global studentDetails to get start value
  set global defaultButtonColor to BTN_Tutor1.BackgroundColor
  set global selectedButtonColor to 
  set global selectedTutorIndex to 1
  set LBL_StudentInfo.Text to "Fetching Student Details..."
  set BTN_Tutor1.Text to "Searching for tutors..."
  set BTN_Tutor2.Text to "Searching for tutors..."
  set BTN_Tutor3.Text to "Searching for tutors..."
  set Web_StudentData.Url to join ("https://script.google.com/macros/s/AKfycbw8hrL7a..."
  call Web_StudentData.Get
  set Web_TutorData.Url to join ("https://script.google.com/macros/s/AKfycbwizdiR..."
  set Clock1.TimerEnabled to true

```

```

when BTN_Tutor2.Click
do
  set LBL_TutorID_For_Matchup.Text to LBL_TutorID2.Text
  set BTN_Tutor1.BackgroundColor to 
  set BTN_Tutor2.BackgroundColor to 
  set BTN_Tutor3.BackgroundColor to 

```

```

when BTN_Tutor3.Click
do
  set LBL_TutorID_For_Matchup.Text to LBL_TutorID3.Text
  set BTN_Tutor1.BackgroundColor to 
  set BTN_Tutor2.BackgroundColor to 
  set BTN_Tutor3.BackgroundColor to 
  set LBL_Debug3.Text to get global selectedTutorID

```

```

when Clock1.Timer
do
  if is a list? thing get global selectedStudentDetails and length of list list get global selectedStudentDetails > 0
  then
    set Clock1.TimerEnabled to false
    call Web_TutorData.Get

```

when **HomeImg** .Click

do open another screen screenName **Home_Page**

when **Notifier1** .AfterChoosing

choice

do if **get choice** = **" Yes "**

then call **UpdateMatchupData**
call **Notifier1** .ShowAlert

notice **" 🎉 Congratulations! Matchup Complete! , You will... "**

open another screen screenName **Home_Page**

else call **Notifier1** .ShowAlert

notice **" ❌ Match Canceled "**

when **Web_TutorData** .GetText

url **responseCode** **responseType** **responseContent**

do if **get responseContent** ≠ **" Tutor Not Found "**
then set **global TutorList** to call **Web_TutorData** .JsonTextDecode
jsonText **get responseContent**

set **global MatchList** to **get global TutorList**

for each **tutor** in list **get global TutorList**

do if **not** **is list empty?** list **get global selectedStudentDetails**

then if **select list item list** **get tutor** index **19** **select list item list** **get global selectedStudentDetails** index **17** **and** **select list item list** **get tutor** index **16**

then **global matchScore** **100**

call **SetTutorButton**
index **get global index**
tutorData **get tutor**
set **global index** to **get global index** + **1**

else if **get global index** ≤ **3**

then if **get global index** ≤ **1**

then set **BTN_Tutor1** .Text to **" No Tutor Found "**

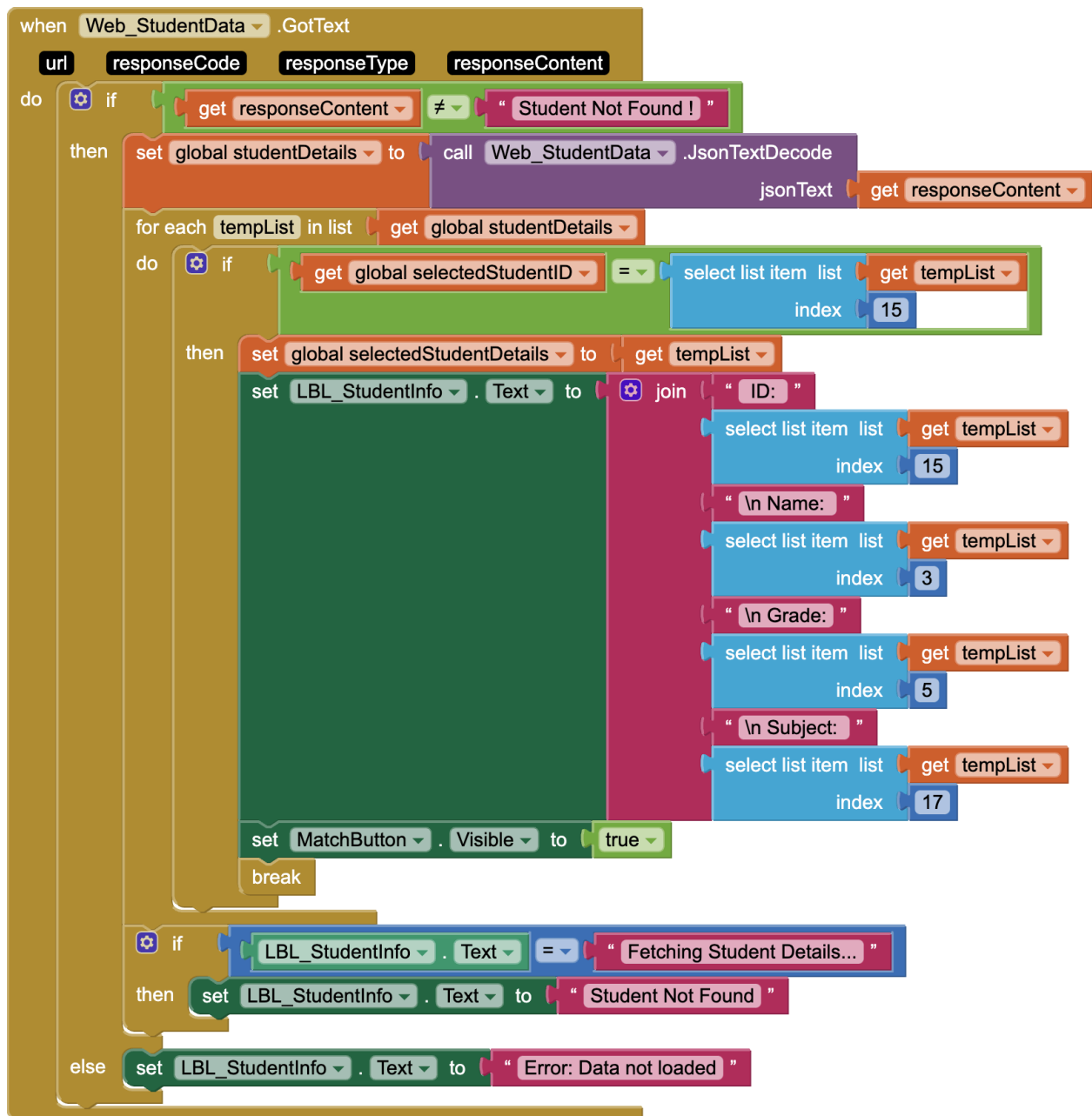
if **get global index** ≤ **2**

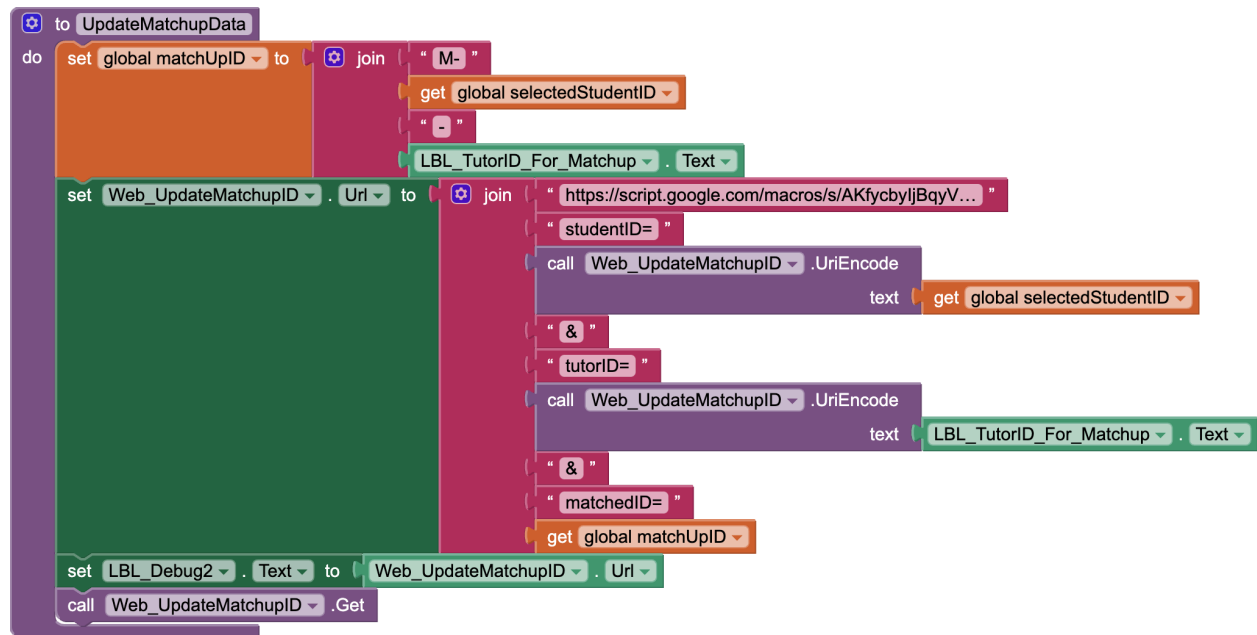
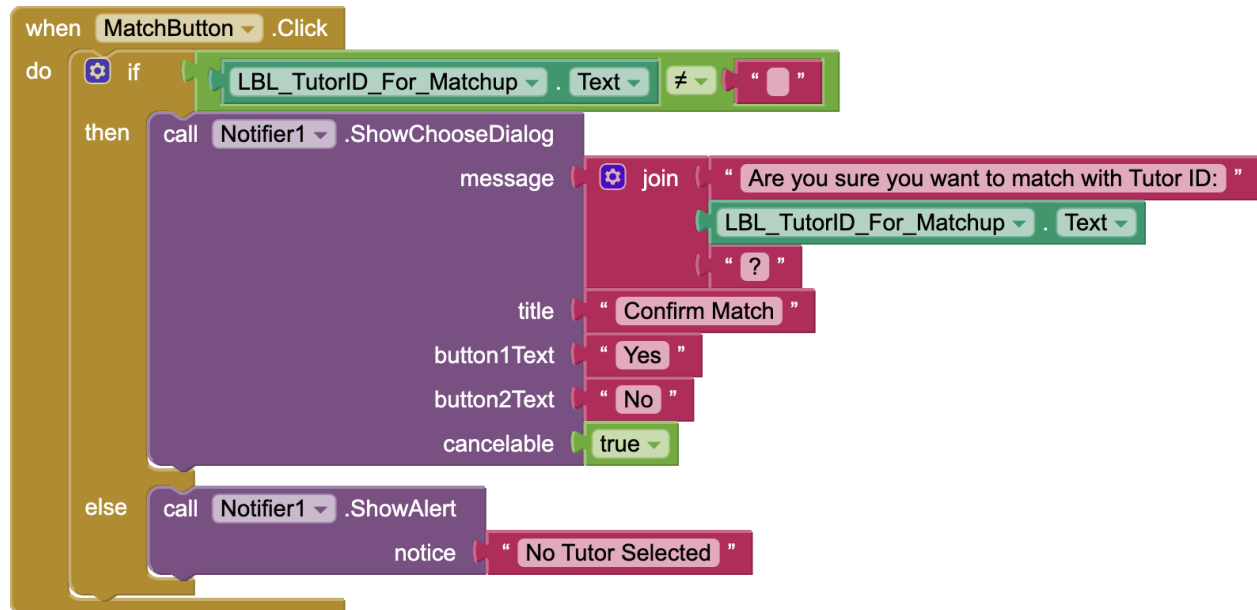
then set **BTN_Tutor2** .Text to **" No Tutor Found "**

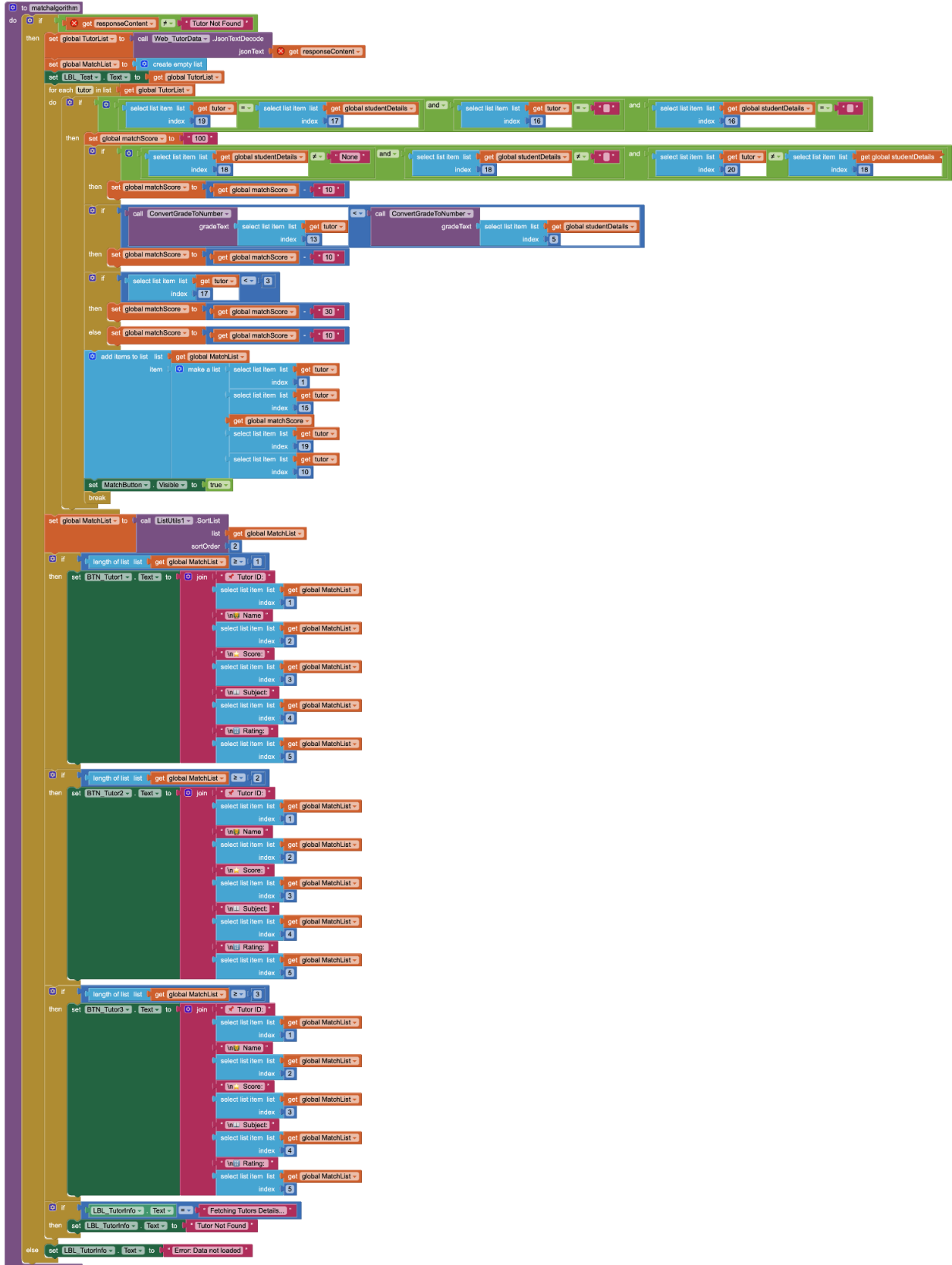
if **get global index** ≤ **3**

then set **BTN_Tutor3** .Text to **" No Tutor Found "**

else







initialize global matchUpID to " "

initialize global defaultButtonColor to 0

initialize global selectedButtonColor to 0

initialize global selectedTutorID to " "

initialize global selectedTutorIndex to 1

initialize global index to 1

initialize global matchScore to " "

initialize global TutorList to " "

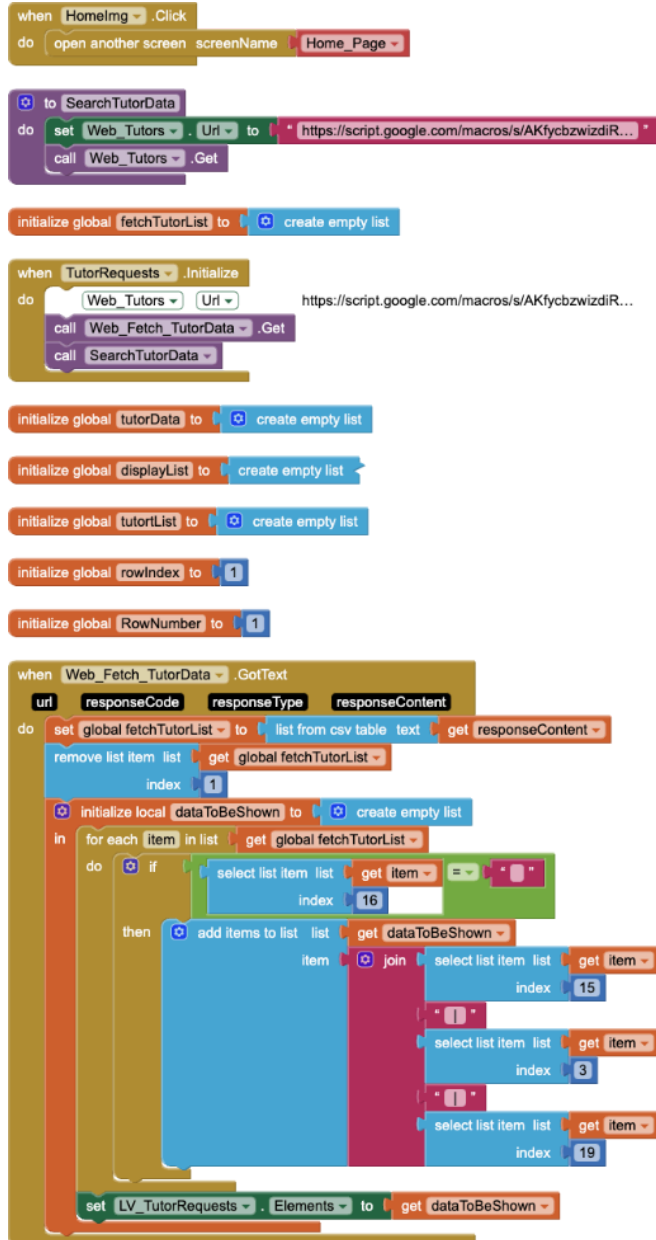
initialize global MatchList to " "

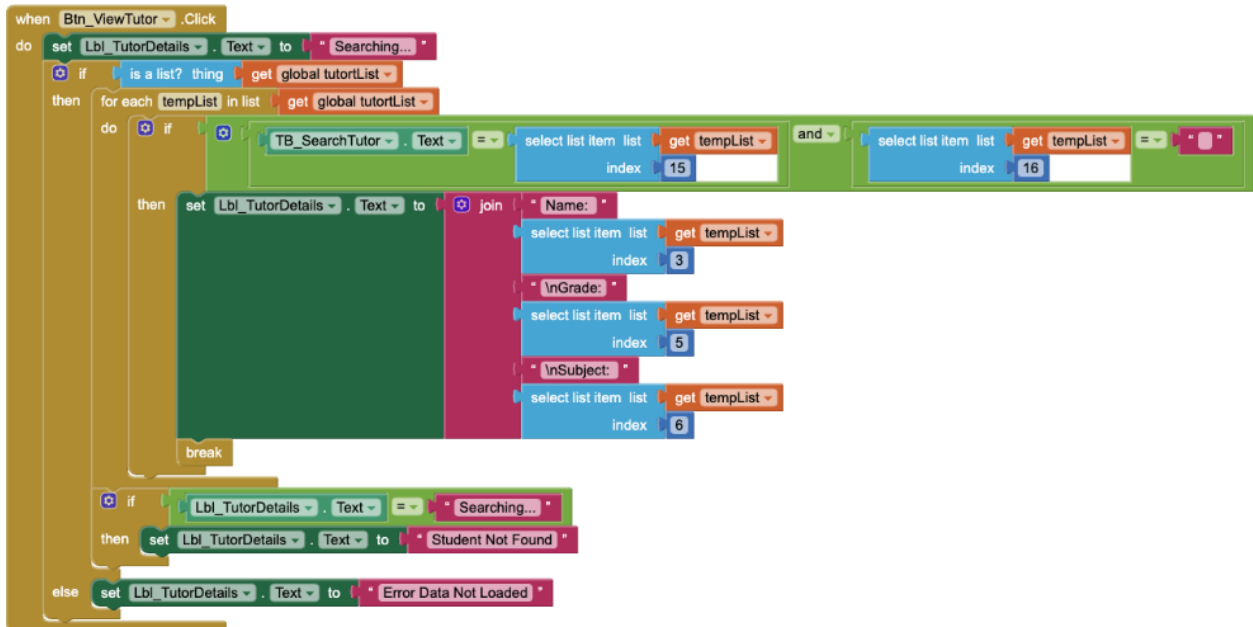
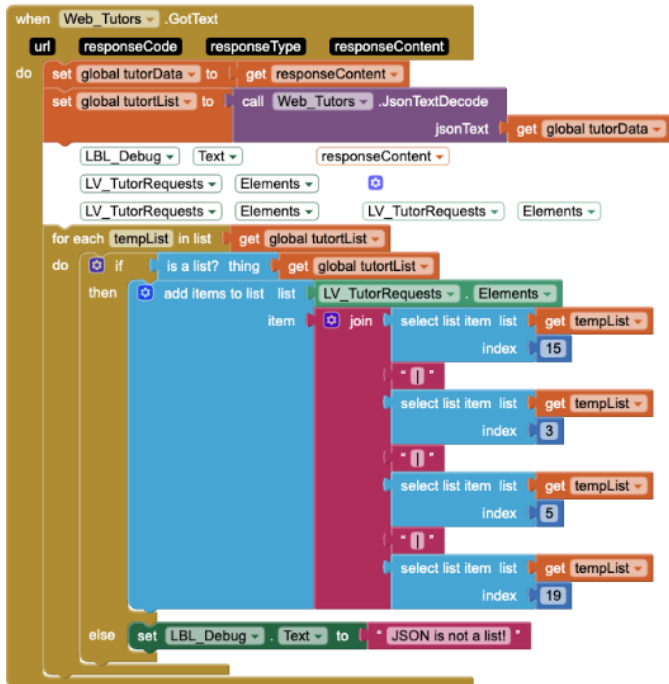
initialize global studentDetails to " "

initialize global selectedStudentDetails to " "

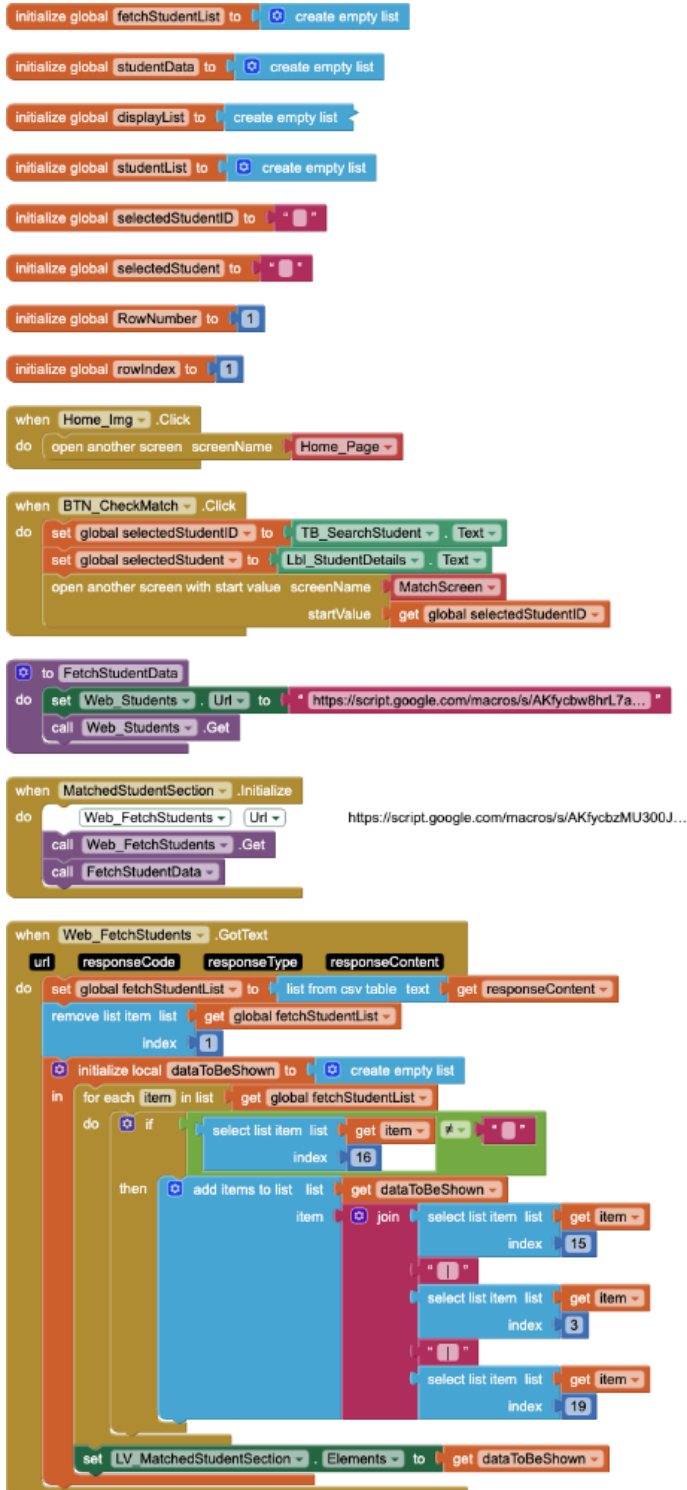
initialize global selectedStudentID to " "

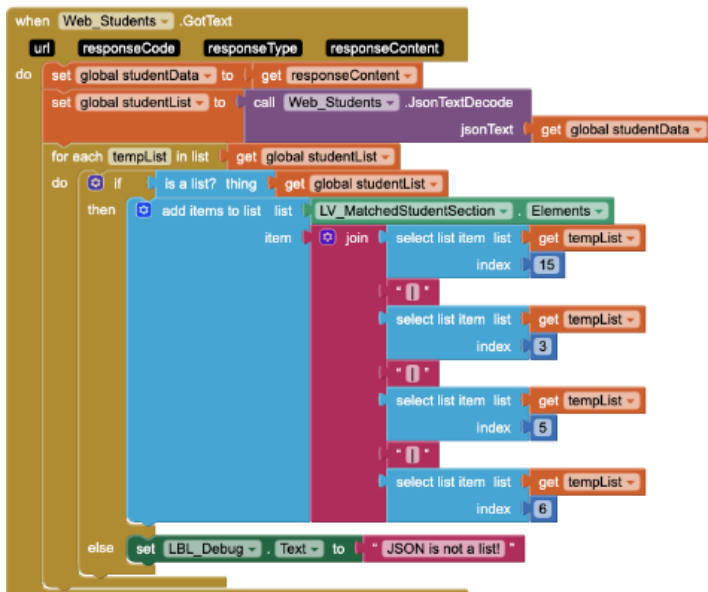
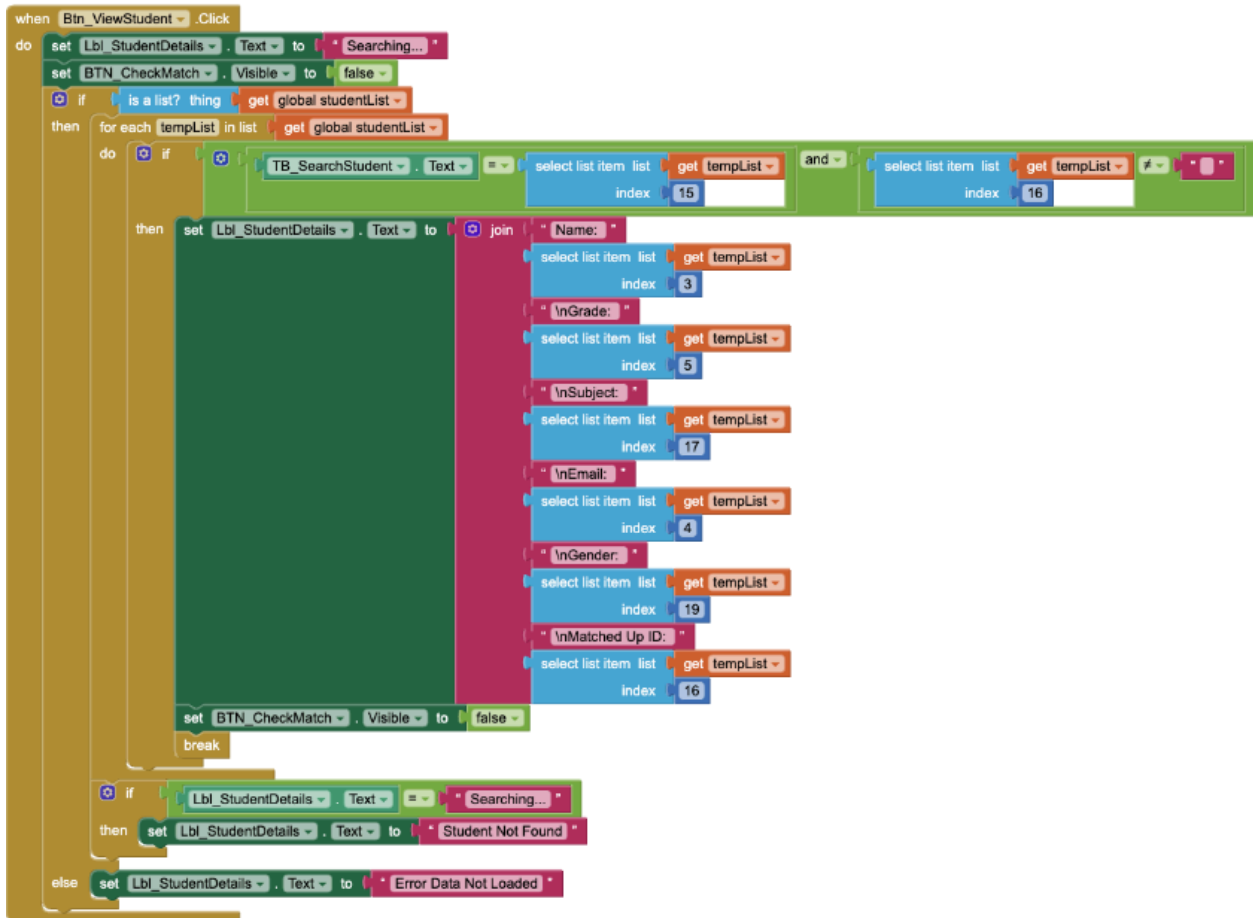
Unmatched Tutors



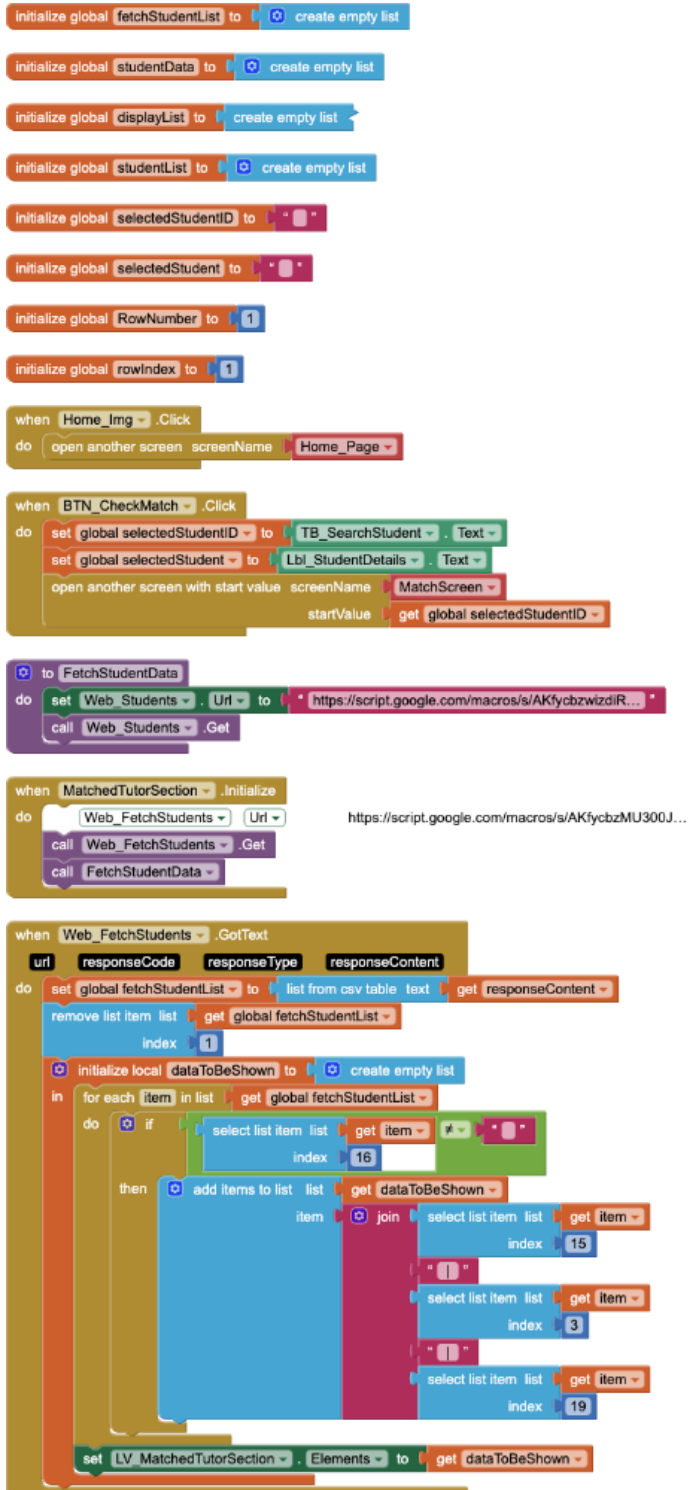


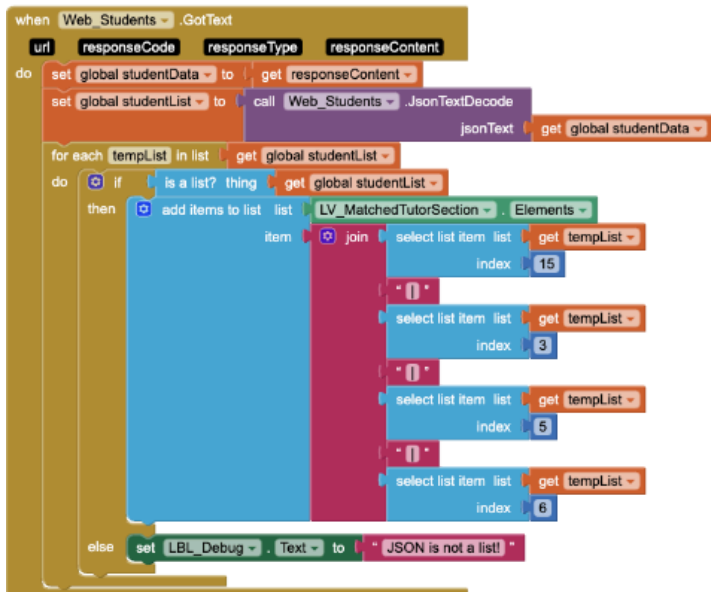
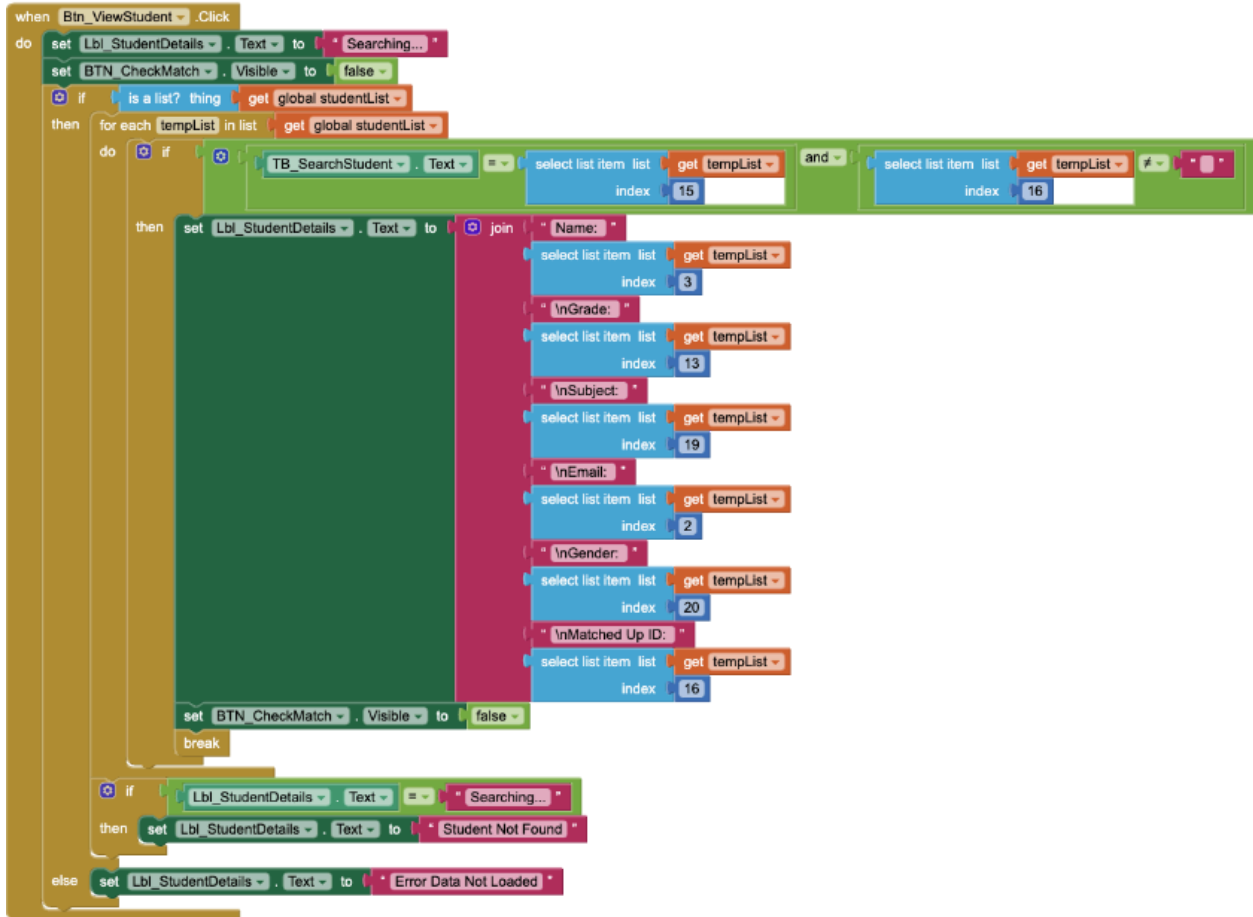
Matched Students





Matched Tutors





Send Bulk Email

```
when Home_Img .Click
do open another screen screenName Home_Page
```

```
when Send_mail_to_student .Click
do
  set Web1 . Url to "https://docs.google.com/spreadsheets/d/1n3XnVY7_..."
  set ActivityStarter3 . Action to "android.intent.action.VIEW"
  set ActivityStarter3 . DataUri to
    join
      "mailto: "
      make a list "ib.csia.testi..."
      "?subject= "
      "Tutoring Assignment Confirmation"
      "&body= "
      "THIS IS AN AUTO GENERATED MAIL REMINDER Dear St..."
  call ActivityStarter3 .StartActivity
```

```
when Send_mail_to_tutor .Click
do
  set Web1 . Url to "https://docs.google.com/spreadsheets/d/1n3XnVY7_..."
  set ActivityStarter4 . Action to "android.intent.action.VIEW"
  set ActivityStarter4 . DataUri to
    join
      "mailto: "
      make a list "ib.csia.testi..."
      "?subject= "
      "Tutoring Assignment Confirmation"
      "&body= "
      "THIS IS AN AUTO GENERATED MAIL REMINDER Dear Tu..."
  call ActivityStarter4 .StartActivity
```

Google Apps Script

Fetch_students_teacher

```
function doGet(e) {
    return getFilteredStudents();
}

function getFilteredStudents() {
    var sheet =
SpreadsheetApp.getActiveSpreadsheet().getSheetByName("New_Student_Section");
    var data = sheet.getDataRange().getValues();

    var students = [];

    for (var i = 1; i < data.length; i++) {
        var row = data[i];
        var matchedId = row[15]; // Matched ID (Column P)

        if (matchedId === "" || matchedId === null) {
            students.push({
                "studentID": row[15] || "", // Ensure empty values are
handled
                "name": row[2] || "",
                "gender": row[5] || "",
                "grade": row[19] || "",
                "subject": row[17] || ""
            });
        }
    }

    return ContentService.createTextOutput(
        JSON.stringify(students, null, 2) // Pretty-print JSON
    ).setMimeType(ContentService.MimeType.JSON);
}
```

Login

```
function doGet(e) {
    var ss =
SpreadsheetApp.openById("1-SwMl6_YQUae5EI8A0D8u-y6XiYWB6Ohluiw9bMR8FU");
// Your Sheet ID
    var sheet = ss.getSheetByName("Login_Credentials"); // Sheet Name for
login

    var email = e.parameter.email;
    var password = e.parameter.password;

    var data = sheet.getDataRange().getValues();
}
```

```

for (var i = 1; i < data.length; i++) {
    if (data[i][0] == email && data[i][1] == password) {
        return ContentService.createTextOutput("success");
    }
}
return ContentService.createTextOutput("failed");
}

```

students

```

function doGet(e) {
    var sheetId = "1PDNTaHJZSPA4nH5y86kMrRh2o0UTY4WtDXxnIeGeMOY"; // Your
    Google Sheet ID
    var ss = SpreadsheetApp.openById(sheetId);

    // Check if 'type' parameter exists in the request
    var sheetName = e.parameter.type;

    if (sheetName) {
        var sheet = ss.getSheetByName(sheetName);
        if (!sheet) {
            return ContentService.createTextOutput("Invalid Sheet
Name").setMimeType(ContentService.MimeType.TEXT);
        }

        var data = sheet.getDataRange().getValues(); // Fetch all data from
        the sheet

        // Remove Header Row (Optional)
        data.shift();

        return
        ContentService.createTextOutput(JSON.stringify(data)).setMimeType(Content
        Service.MimeType.JSON);
    }

    return ContentService.createTextOutput("Missing type
parameter").setMimeType(ContentService.MimeType.TEXT);
}

```

tutors

```

function doGet(e) {
    var sheetId = "1n3XnVY7_00dCnNbpNXVbzrn9uFyLDxK7nOPizodnA1M"; // Your
    Google Sheet ID
    var ss = SpreadsheetApp.openById(sheetId);

    // Check if 'type' parameter exists in the request
    var sheetName = e.parameter.type;

    if (sheetName) {
        var sheet = ss.getSheetByName(sheetName);

```

```

    if (!sheet) {
        return ContentService.createTextOutput("Invalid Sheet
Name").setMimeType(ContentService.MimeType.TEXT);
    }

    var data = sheet.getDataRange().getValues(); // Fetch all data from
the sheet

    // Remove Header Row (Optional)
    data.shift();

    return
ContentService.createTextOutput(JSON.stringify(data)).setMimeType(Conten
tService.MimeType.JSON);
}

    return ContentService.createTextOutput("Missing type
parameter").setMimeType(ContentService.MimeType.TEXT);
}

```

update_matched_tutor_students

```

function doGet(e) {
    var sheetStudents =
SpreadsheetApp.openById("1PDNTaHJZSPA4nH5y86kMrRh2o0UTY4WtDXxnIeGeMOY").
getSheetByName("New_Student_Section");
    var sheetTutors =
SpreadsheetApp.openById("1n3XnVY7_00dCnNbpNXVbzrn9uFyLDxK7nOPizodnA1M").
getSheetByName("New_Tutor_Section");

    var studentID = e.parameter.studentID;
    var tutorID = e.parameter.tutorID;
    var matchID = e.parameter.matchedID;

    if (studentID && tutorID && matchID) {
        updateMatchup(sheetStudents, studentID, matchID);
        updateMatchup(sheetTutors, tutorID, matchID);
        return ContentService.createTextOutput("✅ Matchup Updated
Successfully");
    }
    return ContentService.createTextOutput("❌ Error: Missing
Parameters");
}

function updateMatchup(sheet, searchID, matchID) {
    var data = sheet.getDataRange().getValues();
    var headers = data[0]; // Get column headers
    var idColumn = headers.indexOf("StudentID") > -1 ?
headers.indexOf("StudentID") : headers.indexOf("TutorID");
    var matchColumn = headers.indexOf("MatchedID");

    if (idColumn === -1 || matchColumn === -1) {
        return; // Stop execution if headers are incorrect
    }
}

```

```
}  
  
for (var i = 1; i < data.length; i++) {  
    if (data[i][idColumn] == searchID) {  
        sheet.getRange(i + 1, matchColumn + 1).setValue(matchID);  
        break;  
    }  
}  
}
```